ПРАКТИЧЕСКАЯ РАБОТА «РАБОТА С DOM. ПОИСК ЭЛЕМЕНТОВ, ИЗМЕНЕНИЕ КОНТЕНТА И СТИЛЕЙ, СОЗДАНИЕ УЗЛОВ»

***Цель**: закрепить навыки манипуляции DOM-деревом: поиск элементов (querySelector), безопасное управление контентом (textContent), работа с классами (classList) и динамическое создание узлов (createElement). Подготовка базы для будущих SPA-проектов (курсовых), структурировать код без «спагетти-кода»*

Подготовка проекта (как в реальной работе)

1. Создайте папку проекта: `js-practice-06`
2. Откройте папку в VS Code.
3. Чтобы не тратить время на CSS, используйте готовую верстку из файла `index.html` и оформление из файла `style.css`. Основная работа выполняется в файле: `script.js`.
4. Создайте репозиторий GitHub с названием `js-practice-06` и загрузите туда проект.

Структура репозитория

```
js-practice-06/
├── index.html
├── style.css
├── script.js
├── screenshots/
│   └── (ваши скриншоты)
└── README.md
```

В **README.md** кратко опишите:

- Название проекта
- Используемые технологии (HTML, CSS, JS)
- Ответы на контрольные вопросы (смотри соответствующий раздел)

Общие требования

- Всё JS только в script.js (**никаких inline-атрибутов**).
- **Комментарии «ПОЧЕМУ?»**: Возле каждого ключевого метода (`createElement`, `append`, `remove`, `classList.toggle`, `textContent`) **ОБЯЗАТЕЛЬНО** должен быть комментарий: `// Использую createElement, потому что...` или `// textContent здесь нужен для защиты от XSS....`
- **ТАБУ:** Запрещено использовать `innerHTML` для вставки данных, которые ввёл пользователь. Нарушение = работа не принята.
- **Валидация:** Пустые карточки добавляться не должны.
- **События:** Только `addEventListener`. Никаких `onclick` в HTML.
- Валидация пользовательского ввода - обязательна.
- Для сравнения использовать `===` и `!==`.
- Обязательная защита: при удалении всех элементов — подтверждение `confirm()`.
- Код аккуратный, понятный, с комментариями. Проект запускается через Live Server; в консоли ошибок быть не должно.
- README.md: название проекта, короткое описание (цель + используемые технологии), ответы на контрольные вопросы и инструкции по запуску.

Концепция мини-проекта

Вы реализуете **интерактивную витрину карточек услуг** (упрощённая модель будущего SPA).

Каждая карточка содержит:

- Название
- Краткое описание
- Категорию
- Кнопку «Удалить»

Структура страницы (обязательный минимум)

1. Секция заголовка

- `h1`
- `p` — краткое описание проекта

2. Секция «Создать карточку»

Поля:

- `input#title-input` — название (обязательно)
- `textarea#desc-input` — описание
- `select#category-select`
  Варианты:
  - Дизайн
  - Разработка
  - Маркетинг
- `button#add-btn` — Добавить
- `button#clear-btn` — Очистить форму

3. Секция управления отображением

- `button#toggle-theme` — Переключить тему
- `button#highlight-expensive` — Подсветить карточки категории "Разработка"
- `span#card-count` — количество карточек

4. Секция каталога

- `div#card-container`

Карточки создаются **только через JS**.

Функциональные требования

1. Поиск элементов

Использовать:

- `document.querySelector`
- `document.querySelectorAll`

Никаких `getElementById`.

2. Добавление карточки

При клике на «Добавить»:

1. Проверить:
    - Название не пустое
    - Длина минимум 3 символа
2. Если ошибка — вывести сообщение под формой
3. Если всё корректно:
    - Создать DOM-узлы через `createElement`
    - Вставлять пользовательский текст только через `textContent`
    - Добавить карточку в контейнер
    - Очистить форму

Объект карточки (в коде)

```
{
  id: Date.now(),
  title: "...",
  description: "...",
  category: "..."
}
```

(Даже если массив не используется активно — структура должна быть.)

3. Удаление карточки

Каждая карточка должна содержать кнопку «Удалить».

При нажатии:

- карточка удаляется через `element.remove()`
- счётчик обновляется

⚠️ Делегирование пока НЕ обязательно (это будет в теме 7).

4. Работа с классами

Использовать:

- `classList.add`
- `classList.remove`
- `classList.toggle`

Реализовать:

Переключение темы

Кнопка добавляет класс `dark-theme` на `body`.

Подсветка категории

Кнопка `highlight-expensive`:

- Находит все карточки категории "Разработка"
- Добавляет им класс `highlight`

5. Динамический счётчик

`span#card-count` должен:

- показывать актуальное количество карточек
- обновляться после добавления и удаления

ЗАДАНИЯ ПО УРОВНЯМ СЛОЖНОСТИ

Уровень	Задания
🟢 БАЗА - для тех, кому тяжело (минимальные требования)	Задание 1 — динамический список (создание/удаление)
🟡 JUNIOR - стандартный уровень (рекомендуется всем)	Задание 2 — управление стилями и фильтрация
🔴 PRO - для сильных студентов (дополнительные челленджи)	Задание 3 — массовая вставка и работа с шаблонами

Выберите уровень в зависимости от вашей подготовки:

🟢 УРОВЕНЬ «БАЗА» (Для тех, кому тяжело)

Минимальные требования для зачёта.

Задание 1. Интерактивный список услуг

Реализуйте форму добавления карточек услуг в контейнер `#services-container`.

Требования:

- Поле ввода названия услуги и кнопка «Добавить».
- При клике создается элемент `div.card`, внутри которого заголовок `h3` и кнопка «Удалить».
- **Валидация**: название не может быть короче 3-х символов.
- **Удаление**: при нажатии на кнопку «Удалить» карточка должна исчезать из DOM (`remove()`).
- **Счётчик**: на странице должен быть элемент `#total-count`, обновляющийся при каждом действии.

Подсказка — структура кода:

```
const input = document.querySelector('#service-name');
const btn = document.querySelector('#add-btn');
const container = document.querySelector('#services-container');
const counter = document.querySelector('#total-count');

function updateCounter() {
    // Использую querySelectorAll().length, чтобы... (ваш комментарий)
    counter.textContent = container.querySelectorAll('.card').length;
}

btn.addEventListener('click', () => {
```

```
    const title = input.value.trim();
    if (title.length < 3) return alert("Слишком коротко!");

    // Создаю узлы
    const card = document.createElement('div');
    card.classList.add('card');

    const h3 = document.createElement('h3');
    h3.textContent = title; // ПОЧЕМУ textContent? (ваш комментарий)

    const delBtn = document.createElement('button');
    delBtn.textContent = "Удалить";

    // Логика удаления
    delBtn.addEventListener('click', () => {
        card.remove(); // Использую remove, так как...
        updateCounter();
    });

    card.append(h3, delBtn);
    container.append(card);

    input.value = '';
    updateCounter();
});
```

УРОВЕНЬ «JUNIOR» (Стандарт)

Рекомендуемый уровень для всех.

Задание 2. Темы и «Избранное»

Добавьте функционал управления внешним видом.

Требования:

- **Смена темы:** кнопка «Переключить тему», которая через `classList.toggle('dark-mode')` меняет оформление всей страницы (стили в `style.css` уже готовы).
- **Акцент**: добавьте в карточку кнопку «В избранное». При клике на неё карточке добавляется класс `.highlight`, меняющий фон.
- **Фильтр (имитация)**: кнопка «Показать только избранные». При нажатии все карточки без класса `.highlight` должны получить `display: none` (или класс `.hidden`).

ВОПРОСЫ ДЛЯ ИНТЕРВЬЮ (В README.md)

1. Чем `querySelector` отличается от `querySelectorAll`? Что возвращает второй метод?
2. Почему в этой работе мы отдаем предпочтение `textContent`, а не `innerHTML`?
3. Как работает метод `classList.toggle()`? Опишите логику.
4. Что такое «узел» (node) и «элемент» (element) в контексте DOM?
5. Зачем нужен `DocumentFragment` и в каких случаях его использование оправдано?

Скриншоты (папка screenshots/)

- Добавление 3-х разных карточек.
- Результат удаления одной карточки (с обновленным счетчиком).
- Страница в «Тёмной теме».
- Карточка с примененным классом `.highlight`.
- DevTools с изменённым DOM

Критерии оценки (баллы/чек-лист)

- Проект запускается (Live Server), нет ошибок в Console при обычном сценарии.
- 0% — проект не запущен / нет кода.
- 50% — минимальный функционал (**БАЗА**): валидация, нет ошибок.
- 75% — **JUNIOR**: README с корректными ответами + папка screenshots.
- 100% — **PRO**: все JUNIOR + хорошие комментарии «ПОЧЕМУ?» + чистый код .

Самостоятельная работа (необязательно, но желательно)

🔴 УРОВЕНЬ «PRO» (Для сильных студентов)

Задание 3. Оптимизация и шаблоны

- **DocumentFragment**: реализуйте кнопку «Загрузить демо-прайс» (массив из 50 объектов). Вставка должна происходить через `DocumentFragment` за один раз.
- **Редактирование**: при клике на заголовок карточки он заменяется на `input`, позволяя изменить название «на лету».
- Добавить плавную анимацию появления
- Добавить кнопку «Заполнить демо (100 карточек)»

Как расширить задание в будущем (связь с курсовыми темами)

🔹 Для «Погодного приложения»

- динамический рендер карточек
- условное отображение
- работа с классами

🔹 Для «Менеджера задач»

- создание элементов списка
- удаление
- обновление интерфейса

🔹 Для «Онлайн-записи»

- работа с формой
- валидация
- динамическое обновление UI

🔹 Для «Интернет-магазина»

- карточки товаров
- фильтрация
- подсчёт количества

🔹 Для «Базы образовательного контента»

- динамический рендер данных

- работа со списками

Форма сдачи: Отправьте преподавателю ссылку на GitHub-репозиторий в Slack

Примечание: при необходимости вы можете упростить стили. Оценивается логика, корректность условий, обработка ошибок и аккуратность кода.